TASK 7: Override and Specialise Behaviour

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Which methods are overridden and why

ratePerHour() is overridden in TwoWheeler (10), Car (30) and ElectricCar (inherited 30). The rule for every started hour is written once in Vehicle.calculateFee() and uses each subclass's own rate. In Task 12, EBike overrides calculateFee() because its fee is a flat amount and does not depend on hours, and DeliveryVan overrides calculateFee(entryTime, minutes) because its rate depends on the time of day. A subclass version is needed wherever a subclass's rule is different.

@Override and the overriding rules

```
@Override protected double ratePerHour() { return 10; }
```

- Signature: the name and parameter list must match exactly.

- Return type: the same type, or a subtype (covariant return).

- Access: the overriding method cannot be less visible. A protected method can become protected or public, not private.

- @Override asks the compiler to check these rules, so a typo becomes a compile error rather than a silent new method.

Where final is used

- Field regNo is final: a vehicle's registration number must never change after creation.

- FLAT_FEE in EBike is static final: one fixed value shared by all e-bikes.

- A method marked final cannot be overridden. We did not mark calculateFee() final, because EBike and DeliveryVan need to change it. A final method would be the right choice for a rule that must never change, such as the rounding rule, if a subclass should not alter it.

- A class marked final cannot be extended. ElectricCar is not final, so a future subclass could extend it.

equals(), hashCode() and toString()

equals() compares the registration number only, so two Vehicle objects with the same number are equal, even if their types differ. The rule hashCode() must match is also followed: equal objects return the same hash code, so a HashMap can find them. toString() is not overridden in our code yet. If it were, it would return a readable line such as Car TN02B2, which would make printing tickets clearer.

```
@Override public boolean equals(Object o) {
    return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
}
@Override public int hashCode() { return regNo.hashCode(); }
```

Alternative: one calculateFee() with a switch on a type field

A single method could hold the type as a field and use a switch to choose the rate. Time per fee calculation is O(k) in the worst case for an if-else chain, or about O(1) for a switch implemented with a hash, but the switch is still a choice made at every call. Overriding gives O(1) virtual dispatch to the correct subclass. When a new vehicle type is added, the switch method must change in every place it is used. With overriding, one new subclass is written and no existing class changes (only the factory switch that creates the object, which is one place). Space is the same in both designs, because one type field

replaces the subclass identity.